

Universidad
Rey Juan Carlos

Escuela Técnica Superior de Ingeniería Informática

Grado en Matemáticas
Curso 2025-2026

Diseño y Análisis de Algoritmos

PRÁCTICA 1: ALGORITMO VORAZ

Autores: GRUPO 1. Daniel Rodríguez Fernández, Diego Caballero Sánchez

Marzo 2026

Resumen

Este trabajo consiste en el planteamiento, la implementación y el análisis de un algoritmo voraz...

Palabras clave:

- Voraz
- Algoritmos
- Actividades
- Java
- Complejidad

Índice general

1. Introducción **1**

2. Objetivos **3**

3. Descripción y resolución del problema **4**

4. Decisiones de diseño **9**

5. Análisis de complejidad **11**

6. Ejemplos de funcionamiento **14**
6.1. Modo de Ejecución 1: Interfaz por Consola (Interactivo) 14
6.2. Modo de Ejecución 2: Procesamiento por Lotes (Ficheros) 15
6.3. Robustez y Gestión de Errores (Excepciones) 17

7. Conclusiones **19**

8. Trabajos futuros **20**

1

Introducción

El presente documento detalla el diseño, la implementación y el análisis de una solución computacional para el problema de la Selección de Actividades, un caso clásico en el ámbito de la optimización combinatoria. Esta práctica se enmarca dentro de la asignatura de **Diseño y Análisis de Algoritmos**, con el objetivo de profundizar en la aplicación de estrategias voraces.

El problema consiste en maximizar el número de actividades compatibles que pueden hacer uso de un recurso compartido, dado un conjunto de tareas con tiempos de inicio y fin definidos. A diferencia de otras aproximaciones que podrían requerir técnicas más costosas como la programación dinámica, este escenario permite alcanzar una solución óptima global mediante una serie de decisiones óptimas locales, siempre que se seleccione el criterio de ordenación adecuado.

A lo largo de esta memoria, se describirán los siguientes aspectos fundamentales:

- **La Estrategia Voraz:** Justificación del criterio de selección basado en el

tiempo de finalización temprana para garantizar la optimalidad global.

- **Análisis de Eficiencia:** Estudio de la complejidad computacional, demostrando que el uso de estructuras de datos adecuadas y algoritmos de ordenación eficientes permite obtener un coste de $O(n \log n)$.
- **Implementación en Java:** Descripción de la arquitectura del software, incluyendo el uso de programación orientada a objetos para representar las actividades y la gestión de flujos de entrada/salida tanto por consola como por ficheros.
- **Robustez:** Detalle de los mecanismos de control de errores y excepciones implementados para asegurar un funcionamiento estable ante entradas de datos no válidas.

En resumen, este trabajo no busca únicamente resolver el problema algorítmico planteado desde un punto de vista matemático. Más allá de cumplir con los requisitos técnicos del enunciado, el desarrollo de este proyecto nos ha permitido enfrentarnos a problemas reales de programación, como la gestión de ficheros y el control de errores de usuario. El objetivo final de esta memoria no es solo presentar un código que funcione, sino demostrar que comprendemos por qué la solución elegida es la más eficiente y cómo hemos estructurado el trabajo para que sea profesional y fácil de mantener.

2

Objetivos

1. **Desarrollo** Desarrollar un algoritmo que seleccione conjuntos de tamaño maximal de entre una lista de actividades que no deben solaparse.

2. **Algoritmos voraces** Aprender sobre la implementación de algoritmos voraces en Java, y en qué casos estos son efectivos.

3. **Análisis de complejidad** Analizar la complejidad temporal del algoritmo desarrollado.

3

Descripción y resolución del problema

Enunciado del problema

La descripción del problema es la siguiente: Consideramos un conjunto A de n actividades $\{a_0, a_1, \ldots, a_{n-1}\}$ que necesitan utilizar un recurso común, por ejemplo una sala de reuniones. El recurso sólo puede ser usado por una actividad en cada momento. Cada actividad tiene un instante de comienzo c_i y un instante de finalización f_i, donde $0 < c_i < f_i < \infty$. Si se selecciona la actividad a_i, se desarrolla en el intervalo semiabierto de tiempo $[c_i, f_i)$. Las actividades a_i y a_j son compatibles si sus intervalos $[c_i, f_i)$ y $[c_j, f_j)$ no se solapan, es decir, si $c_i \geq f_j$ o $c_j \geq f_i$. El problema de selección de actividades consiste en determinar un subconjunto de actividades compatibles cuya cardinalidad sea máxima. Por ejemplo, sea el siguiente conjunto de actividades:

i	0	1	2	3	4	5	6	7	8	9	10
c_i	1	2	0	5	8	5	6	8	3	2	12
f_i	4	13	6	7	12	9	10	11	8	5	14

Un subconjunto S de actividades compatibles es $\{a_2, a_4, a_{10}\}$. Sin embargo, no es un subconjunto de cardinalidad máxima, como lo son $\{a_0, a_3, a_7, a_{10}\}$ y $\{a_9, a_3, a_4, a_{10}\}$.

El objetivo de este trabajo es codificar un algoritmo voraz que encuentre alguna solución óptima al problema (pudiendo haber varias).

Resolución general

Para desarrollar un algoritmo voraz, necesitamos, en primer lugar, plantear una función de selección y una función de compatibilidad. Para la función de compatibilidad, y tal y como se explica en el enunciado del problema, es evidente que basta con comparar el comienzo de un elemento con el final del otro.
Por lo tanto, nos centraremos en encontrar la función de selección. Debido a que buscamos soluciones que sean lo más compactas posibles, sería tentador escoger como función de selección, el elemento que tenga un intervalo más reducido. No obstante, observemos el siguiente ejemplo:

i	0	1	2
c_i	1	9	10
f_i	10	11	19

Notemos que en este caso, la solución correcta es claramente escoger las actividades $\{a_0, a_2\}$. A pesar de que la actividad a_1 es la que tiene un intervalo de tiempo menor, de haberla escogido no habríamos podido escoger ninguna de las otras dos. Por lo tanto, debemos escoger una función de selección distinta.
En su lugar, podríamos escoger siempre la actividad que termine antes en el tiempo ($min(f_i)$). Esto genera máximos a nivel local porque es una manera de .[en]cajar a la izquierda"las actividades.

Siguiendo esta misma lógica, podemos desarrollar el algoritmo de la siguiente manera:

1. Ordenamos las tareas en orden ascendente según f_i, su finalización.
2. Inicializamos la solución con el índice de entrada de la primera actividad en nuestra lista ya ordenada (de estar vacía, simplemente devolveríamos una lista vacía).
3. En cada paso, escogemos la siguiente actividad por orden, y si esta es compatible con la última que escogimos, entonces la añadimos a nuestra solución.
4. El algoritmo termina simplemente cuando hayamos recorrido toda la lista.

De esta manera, nuestro algoritmo en pseudocódigo quedaría así:

Función Selección_Voraz (A: conjunto de actividades): conjunto de índices

{ Entrada: A es el conjunto de actividades, Salida: S es la lista solución }
$A \leftarrow$ ordenar A por f_i (ascendente)
$ult_elegido \leftarrow A[0]$
$S \leftarrow \{\text{índice}(ult_elegido)\}$

Para cada actividad a en A (desde la segunda) **Hacer:**
    Si Tiempo_Inicio(a) $\geq$ Tiempo_Fin($ult_elegido$) **Entonces**
        $S \leftarrow S \cup \{\text{índice}(a)\}$
        $ult_elegido \leftarrow a$
    Fin_Si
Fin_Para
Devolver S
Fin_función

Además, es posible demostrar que este algoritmo siempre halla máximos globales.

Demostración de que el algoritmo halla óptimos globales.

Sea A el conjunto de actividades descritas en el enunciado. Llamamos S al conjunto de actividades devuelto por nuestro algoritmo, y S' a una hipotética solución de máxima cardinalidad, no necesariamente igual a la nuestra. A priori, no tienen por qué tener el mismo número de elementos, por lo que denotamos $|S| := k$, $|S'| := s$. Por hipótesis es claro que $k \leq s$, y queremos demostrar que $k = s$.

Por definición de nuestro algoritmo, los elementos de S están ordenados por tiempo de finalización f_i. Reordenemos también los elementos de S' para que también estén ordenados así. (A los elementos de S los denotaremos por los índices i, y a los de S' por los índices j.)

En primer lugar, es obvio, por cómo funciona nuestro algoritmo, que el primer elemento de S, a_{i_0} cumple $f_{i_0} \leq f_{j_0}$, donde a_{j_o} es la primera actividad en S'. Por lo tanto, como las demás actividades de S' inician después de f_{j_0} (por compatibilidad), es claro que podemos intercambiar a_{j_0} en S' por el primer elemento de S sin afectar a la cardinalidad ni a su condición de solución. Además, asumiendo que podamos intercambiar los $m \in \mathbb{N}$ primeros elementos en S' por los primeros m elementos de S, es fácil ver el paso inductivo para $m + 1$:

El elemento $(m + 1)$-ésimo de S es compatible con los primeros $m + 1$ elementos de S', ya que estos han sido intercambiados por los de S (hipótesis de inducción). Por otra parte, $f_{i_{m+1}}$ es menor que $f_{j_{m+1}}$, porque de lo contrario nuestro algoritmo lo habría escogido en su paso $(m + 1)$-ésimo. Como este a su vez es menor que los comienzos de todas las actividades posteriores en S', por un argumento de compatibilidad, ello significa que $f_{i_{m+1}} \leq f_{j_x} \forall x > m + 1$.

Concluimos, por lo tanto, que $a_{j_{m+1}}$ es intercambiable en S' por $a_{i_{m+1}}$.

Es decir, todas las k primeras actividades de S se pueden cambiar con las primeras k actividades en S'. Por reducción al absurdo, si suponemos $k < s$, entonces, por el argumento anterior, hay al menos una actividad $a_{j_{k+1}} \in S'$ tal que su inicio es posterior a f_{i_k}, y por lo tanto su final también. Pero en ese caso, nuestro algoritmo debería haberla escogido, ya que es compatible con las actividades que sí escogió, y termina después, por lo que era posterior en la lista ordenada. Es decir, nuestro algoritmo paró antes de su propia condición de parada, lo cual es una contradicción. Queda demostrado que $k = s$, y por lo tanto nuestro algoritmo halla máximos globales.

Observación

Todo el argumento de optimalidad expuesto anteriormente puede aplicarse, de forma simétrica, a un algoritmo voraz que seleccione sistemáticamente el elemento con el mayor tiempo de inicio (c_i) en lugar del menor tiempo de finalización (f_i). En dicho escenario, el algoritmo simplemente construiría la solución de derecha a izquierda en la línea temporal, manteniendo las mismas propiedades de convergencia hacia el óptimo global.

4

Decisiones de diseño

Para la implementación del algoritmo en `Java`, se ha optado por una arquitectura orientada a objetos que garantiza la modularidad. El diseño se articula en torno a los siguientes componentes:

1. **Clase `Actividad` :** La clase actividad tiene tres atributos enteros (índice, inicio, fin) y un atributo estático que contabiliza las actividades ya creadas. Aunque inicialmente se consideró el uso de arreglos de enteros, finalmente se optó por definir esta clase propia para dotar al problema de una mayor carga semántica. Esta aproximación ofrece varias ventajas. En primer lugar, el constructor actúa como primer filtro de integridad, impidiendo la instanciación de actividades donde $c_i \geq f_i$. En segundo lugar, se emplea un atributo estático y privado para gestionar un contador autoincremental, asignando a cada objeto su índice de entrada original de forma más automatizada. Además, la clase permite la implementación de varios métodos importantes para el algoritmo, como el método 'compatible'. Por último, al implementar la interfaz `Comparable`, se define más fácilmente el criterio de ordenación

basado en f_i, permitiendo el uso eficiente de `Collections.sort()`.

2. **Clase** `Algoritmo` Se ha definido como una clase abstracta para evitar su instanciación innecesaria. El método `voraz` se implementa con un modificador `static`. Si bien esto podría haberse definido directamente en la clase principal, se ha optado por este diseño para separar más la lógica interna de la interacción con el usuario.

3. **Estructuras de Datos:** Para la gestión de la colección de actividades, se utiliza la clase `ArrayList`. Esta elección proporciona la flexibilidad de una lista dinámica y una integración nativa con los algoritmos de ordenación de la API de Java. Nos permite un recorrido lineal eficiente de complejidad $O(n)$.

4. **Interfaz y Control de Errores:** La interacción con el usuario y la gestión de excepciones de entrada/salida se centralizan en la clase principal `PracticaVoraz`. Esta clase contiene el método `main` y se encarga de coordinar el flujo de ejecución. Delega la computación pesada a las clases mencionadas anteriormente.

5

Análisis de complejidad

1. Fase de Preprocesamiento y Estructuración

En el método `main` de la clase PracticaVoraz, se realiza la lectura de datos y la creación de los objetos de tipo `Actividad`. Este proceso implica:

- Un bucle `for` que itera exactamente n veces (número de tareas).
- En cada iteración, se instancia un objeto `Actividad`. El constructor de dicha clase realiza operaciones de tiempo constante $O(1)$, incluyendo la asignación de un identificador único mediante un atributo estático autoincremental.
- Se añade cada objeto a un `ArrayList`. Puesto que la inserción al final de un `ArrayList` tiene un coste amortizado de $O(1)$, el coste de esta fase de preparación de la entrada es lineal:

$$T_{prep}(n) \in O(n)$$

2. Fase de Ordenación

Para garantizar que la estrategia voraz el algoritmo requiere que los candidatos estén ordenados de forma ascendente según su tiempo de finalización (f_i).

- Se utiliza el método `Collections.sort(opciones)`. Al haber implementado la interfaz Comparable en la clase Actividad, el método utiliza el `compareTo` sobreescrito que compara los atributos fin.

- Internamente, Java emplea el algoritmo `TimSort`.

El coste temporal de la ordenación por comparación en el mejor, promedio y peor caso es:

$$T_{ord}(n) \in O(n \log n)$$

3. Fase de Selección de Actividades

Una vez que la lista opciones está correctamente ordenada, el método voraz de la clase Algoritmo ejecuta la lógica principal:

- Se inicializa la solución añadiendo el índice de la primera actividad de la lista ordenada a `listaIndexEntrada`.

- Se realiza un único recorrido (bucle `for`) desde $i = 1$ hasta $n - 1$.

- En cada paso, se invoca el método `opcion.compatible(anterior)`, el cual realiza una comparación lógica entre los tiempos de inicio y fin de dos objetos Actividad, resultando en un tiempo constante $O(1)$.

El coste de esta fase de selección es, por lo tanto, proporcional al número de elementos:

$$T_{sel}(n) \in O(n)$$

4. Complejidad Temporal Resultante

El coste total $T(n)$ del algoritmo se define como la suma de los costes de sus fases:

$$T(n) = T_{prep}(n) + T_{ord}(n) + T_{sel}(n)$$

$$T(n) \in O(n) + O(n \log n) + O(n)$$

Siguiendo las reglas de la notación asintótica, el término dominante es $n \log n$. En consecuencia, la complejidad temporal final del algoritmo es:

$$T(n) \in O(n \log n)$$

6

Ejemplos de funcionamiento

6.1. Modo de Ejecución 1: Interfaz por Consola (Interactivo)

El programa ha sido diseñado para detectar la ausencia de argumentos en la línea de comandos. En tal caso, se activa un modo interactivo que guía al usuario mediante mensajes por la salida estándar (`System.out`).

Como se ilustra en la figura, el sistema solicita:

1. El número total de actividades a procesar.

2. El par de valores (inicio, fin) para cada una de ellas.

```
run:
Introduzca el numero de tareas:
4
Tarea n◆mero... 0
Inicio: 1
Fin: 3
Tarea n◆mero... 1
Inicio: 2
Fin: 5
Tarea n◆mero... 2
Inicio: 1
Fin: 8
Tarea n◆mero... 3
Inicio: 0
Fin: 1
Tareas seleccionadas: [3, 0]
```

Una vez finalizada la introducción de datos, el algoritmo procesa la lista y devuelve la solución óptima directamente en la consola.

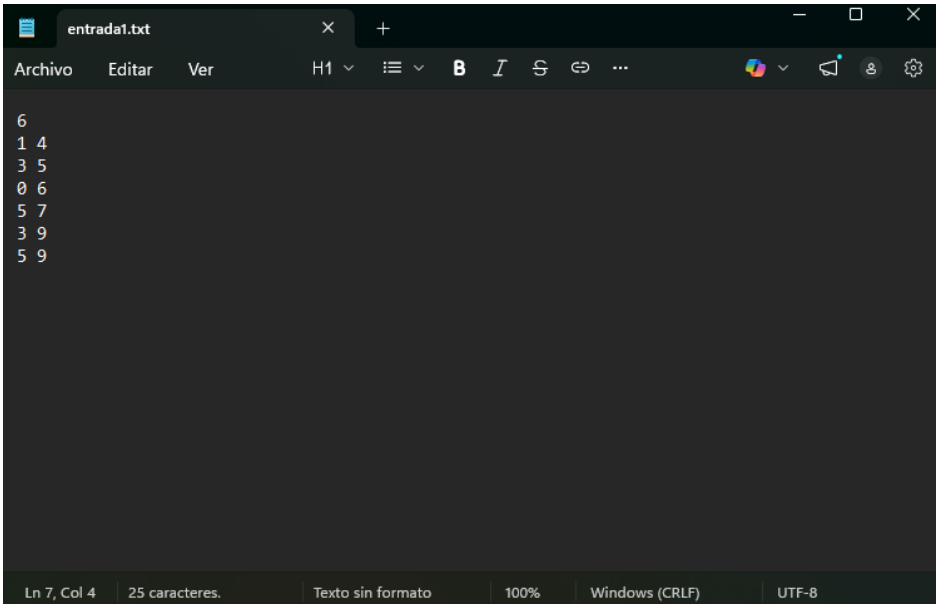
```
Tareas seleccionadas: [3, 0]
```

6.2. Modo de Ejecución 2: Procesamiento por Lotes (Ficheros)

Para escenarios de alta carga de datos, el programa permite el paso de parámetros (`args[0]` y `args[1]`). Los cuales son interpretados como archivo de entrada (args[0]) y de salida (args[1]):

En la siguiente captura se muestra una sesión de la línea de comandos (CMD) donde:

- Se invoca la máquina virtual de Java pasando un fichero de entrada con 11 tareas (`entrada1.txt`).
- El programa finaliza su ejecución sin errores, indicando que la lógica se ha procesado correctamente.




```
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\diego>cd C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz

C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz>"C:\Program Files
\Apache NetBeans\jdk\bin\java.exe" practicavoraz.PracticaVoraz entrada1.txt
Tareas seleccionadas: [0, 3]
```

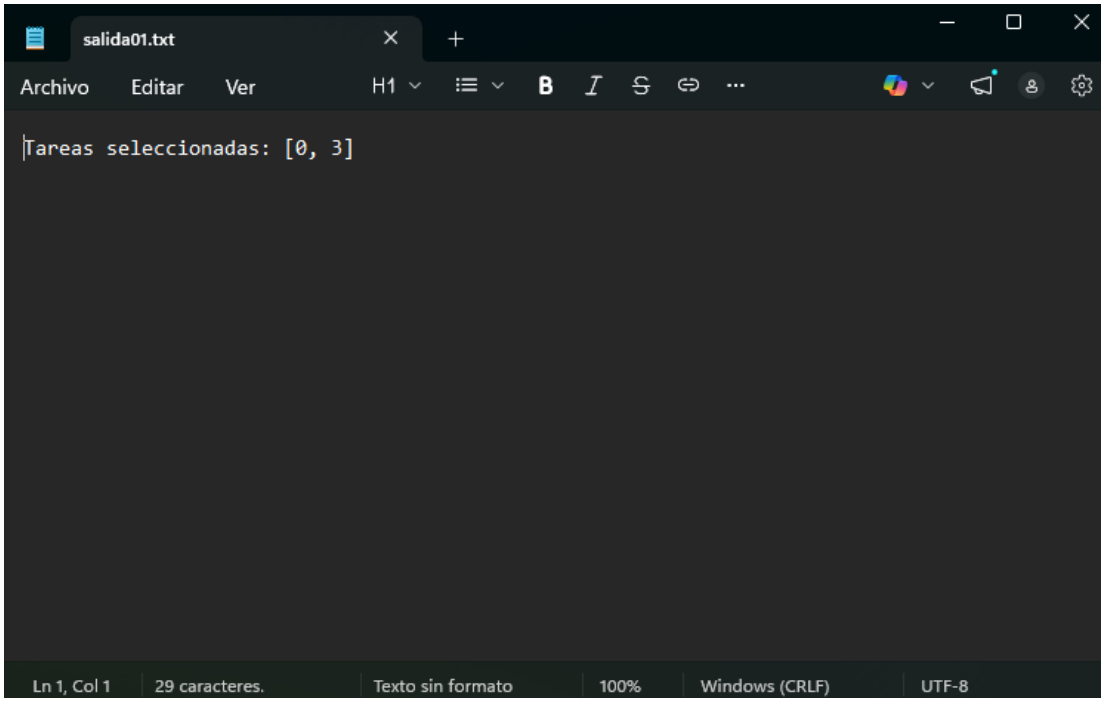
Como salida devuelve en el propio cmd la lista con la solución. La otra opción es introducir un fichero de texto de salida aparte del de entrada. Lo cual se vería:

```
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\diego>cd C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz

C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz>"C:\Program Files\Apache NetBeans
\jdk\bin\java.exe" practicavoraz.PracticaVoraz entrada1.txt salida01.txt
```

De esta forma tal como pide el enunciado se deja la solución en un fichero de texto con el nombre indicado por args[1]:



6.3. Robustez y Gestión de Errores (Excepciones)

Para asegurarnos de prever fallos del entorno o del usuario, se han implementado bloques `try-catch-finally` para capturar errores críticos.

En la siguiente imagen se muestra uno de los controles más relevantes: la protección contra sobrescritura. Si el usuario especifica un nombre de fichero de salida que ya existe en el directorio de trabajo, el programa detiene la ejecución y lanza un mensaje informativo por el canal de error (`System.err`), protegiendo así la integridad de los datos previos.

```
Microsoft Windows [Versión 10.0.26200.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\diego>cd C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz

C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz>"C:\Program Files\Apache NetBeans
\jdk\bin\java.exe" practicavoraz.PracticaVoraz entrada1.txt salida01.txt

C:\Users\diego\OneDrive\Documentos\NetBeansProjects\PracticaVoraz>"C:\Program Files\Apache NetBeans
\jdk\bin\java.exe" practicavoraz.PracticaVoraz entrada1.txt salida01.txt
Error: El fichero de salida 'salida01.txt' ya existe.
```

Los controles de errores más importantes implementados cubren los siguientes

escenarios:

- **Validación de Argumentos y Ficheros:** Se verifica la existencia del fichero de entrada antes de intentar su lectura. En caso de que el archivo no sea accesible, se captura la excepción `FileNotFoundException`, evitando que el programa se detenga de forma inesperada.
- **Control de Formato de Datos:** Durante la lectura, el sistema asegura que las entradas sean estrictamente valores enteros. Si se detectan caracteres alfanuméricos o formatos incorrectos, se lanza una `InputMismatchException` con un mensaje informativo para el usuario.
- **Integridad de los Intervalos:** Se comprueba que el número de tareas sea un entero positivo ($n > 0$). Además, en el constructor de la clase `Actividad`, se valida que el tiempo de inicio sea menor al de fin ($inicio < fin$) y que ambos sean no negativos, lanzando una `IllegalArgumentException` si se detectan intervalos lógicamente imposibles.
- **Protección de Sobrescritura:** Antes de generar el resultado, el programa comprueba si el fichero de salida ya existe en el directorio. Si el archivo está presente, se detiene la ejecución para proteger la integridad de los datos previos del usuario.
- **Gestión de Recursos:** Independientemente del éxito o fallo de la ejecución, el bloque `finally` garantiza el cierre de los flujos de lectura (`Scanner.close()`), liberando los recursos del sistema operativo.

7

Conclusiones

En este trabajo se ha aplicado la estrategia resolutiva de los algoritmos voraces a un ejemplo clásico de selección de actividades. Además, se ha demostrado que esta estrategia siempre encuentra óptimos globales en este caso, y se ha analizado la complejidad del algoritmo, comprobando que esta está acotada inferiormente por el proceso de ordenación, con $O(n\,logn)$. Finalmente, se ha ejemplificado el algoritmo implementándolo en el lenguaje de programación Java, y resolviendo varios ejemplos.

En definitiva, hemos comprobado la utilidad de los algoritmos voraces para resolver problemas que a priori no parecen tener una solución trivial.

8

Trabajos futuros

Si bien nuestro algoritmo actual funciona correctamente, cumpliendo los objetivos de esta práctica, hay varias modificaciones que podrían tener interés de cara a trabajos futuros.

En primer lugar, en lo referente a la interacción con el usuario, sería deseable que el programa tuviera una interfaz más cómoda.

Además, el manejo de errores podría ser menos restrictivo, para evitar que el usuario deba volver a escribir todos los datos en caso de error. Por ejemplo, supongamos que el usuario comete un error, y al introducir por teclado la tercera actividad, confunde el inicio con el fin. Con la programación actual, esto simplemente devolverá una excepción, haciendo que tenga que volver a escribir las anteriores actividades. Para mejorar esto, mediante un bucle `while` que comprobara que los argumentos inicio y fin son correctos, y que pida al usuario reescribirlos hasta que lo sean, el programa podría ser mucho más cómodo para el usuario.

Un mecanismo similar se podría implementar para el fichero de entrada (si genera una excepción, pedir al usuario volver a introducirlo por teclado hasta que no lo haga) y para el fichero de salida.

Otra mejora que se podría hacer al programa actual, es que la solución fuese más detallada. Actualmente, simplemente devuelve una lista de índices, pero por claridad, esto se podría mejorar, imprimiendo una lista de actividades ordenadas por su índice, con inicio y fin. Esto sería relativamente sencillo con una implementación del método `toString()` en la clase `Actividad`, y haciendo que el método `voraz()` devuelva la lista de actividades, en vez de simplemente los índices, o alternativamente, buscando las actividades correspondientes a los índices seleccionados en la lista original.
Por último, podríamos considerar un formato más realista para las actividades. Si bien en esta práctica se representan los tiempos de inicio y de fin con números enteros, en futuros trabajos sería interesante considerar modelarlos con clases como `LocalTime` o `LocalDateTime`, lo cual no alteraría nuestro algoritmo de forma significativa, pero permitiría adaptarlo más a las potenciales necesidades del mundo real.

Bibliografía

[1] Paredes Velasco, Máximo. *Apuntes del Aula Virtual*. Diseño y análisis de Algoritmos, Universidad Rey Juan Carlos.